FinLake

A Real-Time Fraud Detection & Risk Analytics Lakehouse

A hands-on Databricks data engineering project, built to develop production-grade skills and prepare for product-based company interviews

---

Domain: Fintech / Digital Banking • Level: Intermediate • Scope: Batch + Streaming Lakehouse
Built on the Medallion Architecture with Unity Catalog, Delta Lake, Structured Streaming, MLflow, and CI/CD

1. Project overview

FinLake simulates the data platform of a digital bank. It ingests historical transaction records and a live transaction stream, cleans and conforms them into a governed Lakehouse, detects fraud in near real time, and serves the results to dashboards and downstream applications. The goal is not just to move data from A to B — it is to practice every skill a Databricks data engineer is expected to demonstrate at a product-based company: streaming ingestion, slowly changing dimensions, data quality enforcement, governance, orchestration, CI/CD, and performance tuning, all inside one coherent project.

Why this project works well for interview preparation

- It forces both batch and streaming thinking, which is where most candidates are weakest.
- Slowly changing dimensions (SCD2) and fraud-style stream-static joins are extremely common interview whiteboard questions — you will have actually built them, not just read about them.
- Unity Catalog governance and PII masking map directly to "how do you handle compliance/PII" questions asked at fintech, healthtech, and most large product companies.
- Every phase produces an independently demoable artifact, so you can talk about the project in a structured, STAR-format way even if you stop midway.

---

2. Datasets — what to use and why

A single static CSV cannot demonstrate a Lakehouse. Interviewers want to see how you combine a historical batch source, a live stream, and a slowly changing reference dataset — so this project deliberately uses four data sources working together rather than one big dataset.

Primary recommendation

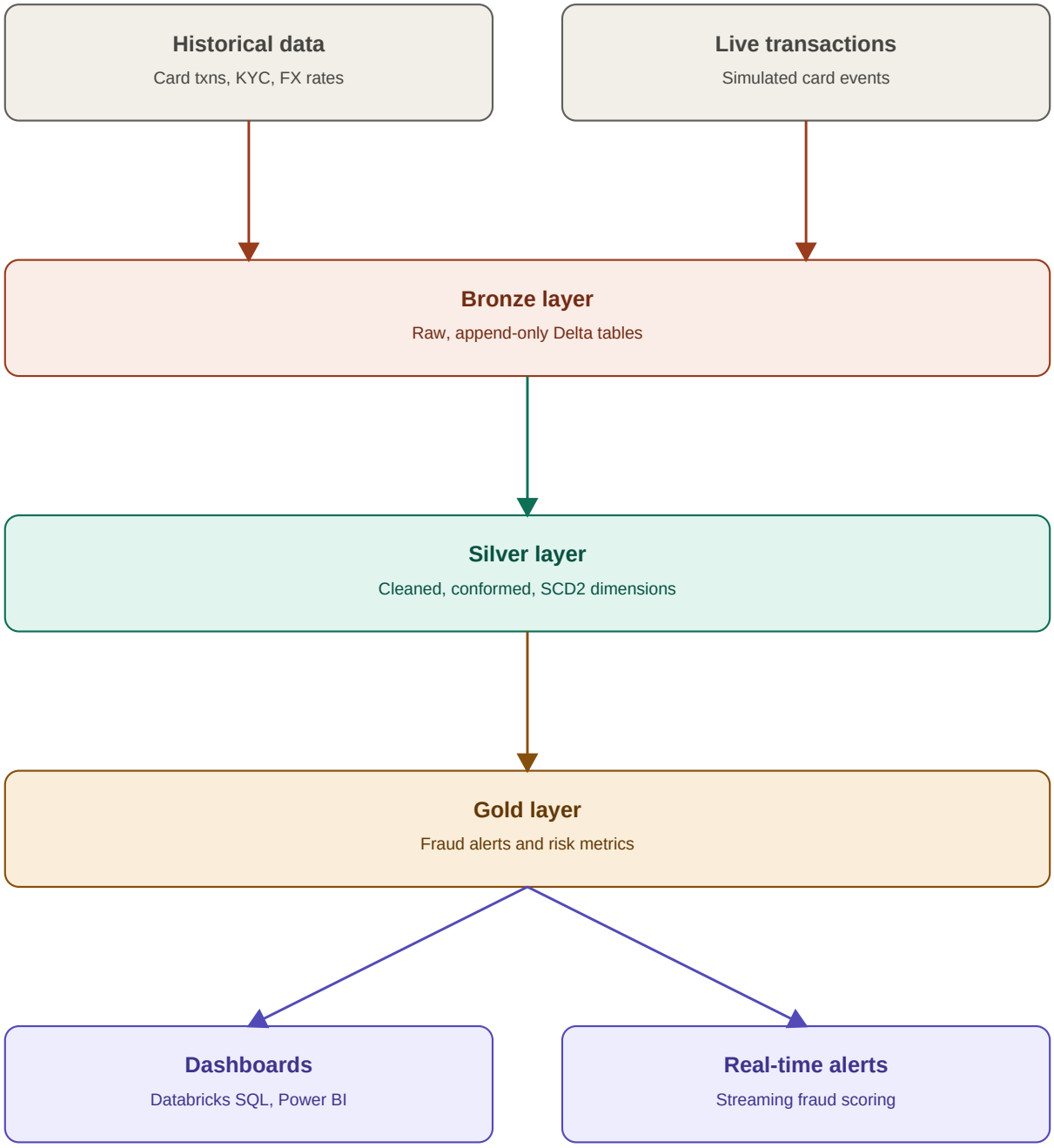
Use the **Kaggle “Credit Card Fraud Detection” dataset** as your anchor dataset. It contains roughly 285,000 real, anonymized European card transactions with a fraud label (Class column), PCA-transformed features (V1–V28), Time, and Amount. It is the most widely recognized fraud dataset in the industry, which matters in an interview — the interviewer immediately understands the data you are referencing without you having to explain a custom schema from scratch. Use it as your historical Bronze source and to train the fraud-detection model in Phase 6.

Dataset	Purpose	Source / format	Why this one
Kaggle Credit Card Fraud Detection	Historical fraud-labeled transactions; ML training	CSV, ~285K rows, Kaggle	Real labeled data; industry-recognized benchmark
Synthetic customer & KYC data	Customer dimension; SCD2 practice	Generated with Python Faker	You control updates, so you can test SCD2 precisely
Synthetic transaction stream	Real-time ingestion source	Python script emitting JSON files (or Kafka)	Mimics production traffic; you control volume and velocity
FX rates	Reference data; currency standardization	Frankfurter API (free, no key)	Real, live external API — practice reference-data ingestion
Lending Club loan data (optional)	Stretch: credit risk module	CSV, Kaggle	Extend the project into credit risk if time allows

Synthetic data schemas to design yourself: customers (customer_id, name, dob, address, kyc_tier, risk_score, updated_at), and transaction events (txn_id, customer_id, card_id, amount, currency, merchant_category, merchant_country, channel, event_time). Keeping these schemas simple but realistic is more valuable than making them comprehensive.

3. Architecture

FinLake follows the Medallion Architecture. Both sources land in Bronze untouched, get cleaned and conformed in Silver (including the SCD2 customer dimension), and are aggregated into business-ready Gold tables that feed dashboards and a real-time alert feed.



Unity Catalog governs every layer (lineage, access control, PII masking). Databricks Workflows or Delta Live Tables orchestrate the pipeline, and Databricks Asset Bundles with GitHub Actions deploy it across dev, staging, and production.

4. Implementation roadmap

Ten phases, each producing an independent, demoable deliverable. Suggested pace: roughly 4–6 weeks at a steady part-time effort (see the timeline in Section 6).

Phase 1 — Foundation & environment setup

Objective: Stand up your workspace, governance structure, and repo before writing any pipeline code.

- Create a Databricks workspace and enable Unity Catalog.
- Create a catalog (e.g. finlake) with three schemas: bronze, silver, gold.
- Create a volume / external location to land raw files.
- Initialize a Git repo and connect it via Databricks Repos.
- Scaffold a Databricks Asset Bundle so every later phase deploys through code.

Deliverable: *A Unity Catalog catalog with bronze/silver/gold schemas, a connected repo, and an empty bundle that deploys successfully.*

Phase 2 — Batch Bronze ingestion

Objective: Load both historical sources as raw, auditable Delta tables.

- Download the Kaggle fraud CSV and upload it to your landing volume.
- Generate 5,000–10,000 synthetic customers with Faker.
- Write a PySpark batch job that writes both sources to bronze Delta tables, adding ingestion metadata (_ingested_at, _source_file).
- Deliberately introduce a schema quirk (a renamed or late-arriving column) to practice schema evolution and the rescued-data column.

Deliverable: *bronze.transactions_historical and bronze.customers_raw Delta tables.*

Phase 3 — Streaming Bronze ingestion

Objective: Ingest a simulated live transaction feed continuously.

- Write a Python producer that emits one JSON transaction event every few seconds to a landing folder (or a lightweight Kafka topic for extra realism).
- Use Auto Loader (cloudFiles) to continuously pick up new files into bronze.transactions_stream.
- Configure checkpointing correctly; restart the stream and confirm no duplicates or data loss.

```
df = (spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", "/Volumes/finlake/bronze/_schemas/txn_stream")
  .load("/Volumes/finlake/bronze/landing/transactions"))

(df.writeStream
  .option("checkpointLocation", "/Volumes/finlake/bronze/_checkpoints/txn_stream")
  .trigger(processingTime="30 seconds")
  .toTable("finlake.bronze.transactions_stream"))
```

Deliverable: *A running Auto Loader stream continuously appending to bronze.transactions_stream.*

Phase 4 — Silver layer & SCD2 customer dimension

Objective: Clean and conform data; build a proper slowly changing customer dimension.

- Dedupe and standardize historical and streaming transactions into a unified silver.transactions table; standardize currency using the FX reference table.
- Build silver.customers_scd2 using MERGE INTO logic tracking effective_start, effective_end, and is_current.
- Simulate a few customer updates (address change, risk-tier change) and confirm SCD2 versions them correctly.

```
MERGE INTO finlake.silver.customers_scd2 AS target
USING customer_updates AS source
ON target.customer_id = source.customer_id AND target.is_current = true
WHEN MATCHED AND target.risk_tier <> source.risk_tier THEN
  UPDATE SET is_current = false, effective_end = source.update_ts
WHEN NOT MATCHED THEN
  INSERT (customer_id, risk_tier, effective_start, effective_end, is_current)
  VALUES (source.customer_id, source.risk_tier, source.update_ts, NULL, true)
```

Deliverable: *silver.transactions and silver.customers_scd2 with verified historical versioning.*

Phase 5 — Streaming aggregation & fraud rules

Objective: Compute near-real-time risk signals on the live stream.

- Apply a watermark and a tumbling window to compute spend velocity per customer (transaction count and total amount per 5-minute window).
- Join the stream against the Silver customer dimension (stream-static join) to bring in risk tier.
- Flag transactions breaching simple rules (e.g. more than 3 transactions in 5 minutes) into gold.fraud_alerts_stream.

```
windowed = (transactions_stream
  .withWatermark("event_time", "10 minutes")
  .groupBy(window("event_time", "5 minutes"), "customer_id")
  .agg(count("*").alias("txn_count"), sum("amount").alias("total_amount")))
```

Deliverable: *A live gold.fraud_alerts_stream table updating within seconds of a suspicious pattern.*

Phase 6 — ML integration with MLflow

Objective: Replace or augment the rules engine with a trained model.

- Train a classifier (e.g. gradient-boosted trees) on the labeled Kaggle fraud data with MLflow autologging.
- Register the best model in the MLflow Model Registry and promote it to "Production."
- Apply the model in batch on silver.transactions, and in streaming via a Pandas UDF inside foreachBatch.

Deliverable: *A registered fraud-scoring model applied to both batch and streaming data, feeding gold.fraud_scores.*

Phase 7 — Orchestration

Objective: Turn your notebooks into a managed, monitored pipeline.

- Convert the Bronze → Silver → Gold logic into a Delta Live Tables pipeline with expectations (e.g. expect amount > 0) so bad records are quarantined automatically.

- Or chain tasks in a Databricks Workflow with dependencies, retries, and failure notifications.

Deliverable: *A scheduled, monitored pipeline that runs unattended and surfaces data-quality failures.*

Phase 8 — Governance & security with Unity Catalog

Objective: Apply the access controls a real bank would require.

- Create dynamic views that mask PII (e.g. only the last 4 digits of a card number) for analyst roles, with full visibility for compliance roles.
- Apply column-level and row-level security based on group membership.
- Enable lineage and confirm you can trace a Gold metric back to its Bronze source visually.

```
CREATE VIEW finlake.gold.transactions_masked AS
SELECT
  txn_id, customer_id, amount, currency,
  CASE WHEN is_member('compliance_team') THEN card_number
       ELSE concat('****-****-****-', right(card_number, 4)) END AS card_number
FROM finlake.silver.transactions
```

Deliverable: *Role-based, column-masked views with confirmed lineage in Unity Catalog.*

Phase 9 — CI/CD with Databricks Asset Bundles

Objective: Deploy the whole project through code, across environments.

- Define dev/staging/prod targets in your bundle's databricks.yml.
- Set up a GitHub Actions workflow that validates and deploys the bundle on every push to main.
- Add a basic test (e.g. pytest on a transformation function) that runs in CI before deployment.

Deliverable: *A green CI pipeline that deploys your DLT pipeline / workflow to a staging catalog automatically.*

Phase 10 — Performance & cost tuning

Objective: Make the pipeline efficient, not just correct.

- Apply Z-ordering or Liquid Clustering on transactions by customer_id.
- Schedule OPTIMIZE and VACUUM jobs.
- Compare a broadcast join vs a shuffle join on the customer dimension and measure the difference in the Spark UI.
- Move dashboard queries to a serverless SQL warehouse and compare cost/latency against an all-purpose cluster.

Deliverable: *A documented before/after performance comparison you can describe in an interview.*

5. Depth topics this project forces you to practice

- Auto Loader: file-notification mode, schema evolution, the rescued-data column
- Structured Streaming: watermarking, windowed aggregation, stream-static joins, foreachBatch
- Delta Lake internals: MERGE, Change Data Feed, time travel, OPTIMIZE / Z-order / Liquid Clustering, VACUUM
- Unity Catalog: catalogs/schemas, lineage, dynamic views for PII masking, row/column-level security
- Orchestration: Delta Live Tables expectations, Databricks Workflows (retries, SLAs, alerts)
- MLflow: tracking, model registry, batch and streaming inference
- CI/CD: Databricks Asset Bundles, GitHub Actions, environment promotion
- Performance tuning: AQE, partition pruning, broadcast joins, Photon, cluster policies vs serverless SQL warehouses

6. Suggested timeline

Week	Focus
Week 1	Phase 1–2: environment setup, batch Bronze ingestion
Week 2	Phase 3–4: streaming Bronze, Silver layer, SCD2 dimension
Week 3	Phase 5–6: streaming aggregation, fraud rules, MLflow model
Week 4	Phase 7–8: orchestration, Unity Catalog governance
Week 5–6	Phase 9–10: CI/CD, performance tuning, README, demo recording

7. Interview question mapping

Each component of this project maps directly to a question theme that comes up repeatedly in product-based company data engineering interviews.

Project component	Interview question theme
SCD2 customer dimension	How do you handle slowly changing dimensions in Spark?
Streaming + watermarking	How do you handle late-arriving data and exactly-once semantics?
Medallion architecture	Walk me through how you would design a Lakehouse for a given domain.
Unity Catalog PII masking	How do you handle data governance and PII in a regulated industry?
OPTIMIZE / Z-order / Liquid Clustering	How do you tune a Delta table that has gotten slow?
MLflow batch + streaming scoring	How would you operationalize an ML model inside a pipeline?
Asset Bundles + GitHub Actions	How do you deploy Databricks pipelines across environments?

8. Next steps

Push the project to a public GitHub repo with a clear README, an architecture diagram, and a short demo recording or GIF of the live fraud alerts updating. For each phase, prepare a one-paragraph STAR-format story (Situation, Task, Action, Result) — this is what turns a side project into a strong interview narrative, rather than just a bullet point on a resume.